Caso de Estudio Report Cruz Roja

Anyela Lineth Cabrera Ordoñez
202540031
anyela.cabrera@correounivalle.edu.co

Carlos Anibal Ceron del Castillo
202537097
carlos.ilich.ceron@correounivalle.edu.co

Javier Felipe Contreras Díaz
202535936
contreras.javier@correounivalle.edu.co

Facultad de Ingeniería, Universidad del Valle

750015C: Fundamentos de Programación Orientada a Objetos

Victor Andres Bucheli Guerrero

19 de marzo de 2026

Preguntas de Autocalificación, Caso de Estudio Report Cruz Roja

1. Abstracción: Revisar las abstracciones del Sistema de Gestión de Donantes de Sangre de la Cruz Roja. Se puede asegurar que las abstracciones permiten cumplir con los principios de responsabilidad única y abierto cerrado?

No, las abstracciones no cumplen con los principios de responsabilidad única. La clase BloodDatabase tiene varias funciones, ya que pide información al usuario, guarda los datos de cada donante en un archivo de texto, busca y elimina donantes; adicionalmente maneja interfaz de usuario con métodos como limpiar la consola. La clase donor contiene un método trim, el cual no hace parte de la abstracción de la clase, para su correcto uso debería incluirse en una clase con herramientas para modificar strings. Por otro lado, la clase Reports tiene una abstracción correcta, encargándose de realizar cálculos estadísticos sobre la información de objetos tipo donor y colaborando con la clase BloodDatabase.

Teniendo en cuenta el principio de abierto-cerrado, en la clase BloodDatabase, los métodos de búsqueda y los menús que se muestran al usuario están escritos de forma fija; por tanto, si en el futuro se desea añadir nuevos departamentos, tipos de sangre o modificar los criterios de búsqueda, se tendrá que modificar el código de la clase en lugar de solamente extenderlo. Lo que significa que tampoco cumple con este principio.

2. Encapsulamiento: Revisar el encapsulamiento de la clase Donor. Se puede asegurar que la información crítica sobre los donantes esté protegida y se manipule de forma segura dentro del sistema, evitando modificaciones indebidas y asegurando la integridad de los datos en el sistema?

La clase donor tiene algunos problemas de encapsulamiento, a pesar de que el método trim se encuentra correctamente declarado como privado, ya que este no hace parte de la interfaz de la clase, los atributos de cada donante (donorId, district, bloodType, age, name, address, number) se encuentran definidos como públicos, por lo cual se pueden acceder y modificar de manera externa sin ningún control o proceso de validación.

La clase hace uso de getters (getAge(), getBloodType(), getDistrict()) pero estos no están ofreciendo ninguna protección a los datos y son redundantes, ya que los atributos son públicos. Adicionalmente no existen implementados setters que controlen la lógica de validación para evitar comportamientos indeseados (como la asignación de edades negativas o fuera del rango permitido).

Por esta razón, no se puede asegurar que los datos están protegidos, ni que se mantenga su integridad. Para poder solventar este problema, es necesario mover todos los atributos de la clase a Private, dejando todos los métodos de interfaz incluyendo getters y setters (que controlen el acceso y validen los datos) declarados como públicos.

3. Punto para bonificación: Al finalizar el año se requiere hacer un reporte de cierre de año, en el cual se presente: el total de donantes por tipos de sangre y por las diez zonas del país. El reporte debe calcular el gran total de donantes y su porcentaje correspondiente para cada tipo de sangre. Escriba el reporte en forma de matriz,

donde las filas son los tipos de sangre, con sus respectivo porcentaje, y las columnas las diez regiones del país:



En términos de estadística descriptiva, el reporte presenta una distribución de frecuencias que organiza los datos en una matriz de doble entrada. La tabla permite observar la relación entre la variable cualitativa de tipo de sangre y la variable espacial de departamento de procedencia, donde cada celda indica la frecuencia absoluta de individuos que comparten ambos atributos.
Por lo tanto permite identificar como el tipo de sangre B+ es quién tiene el mayor número de donantes, seguido del A+ y el O-.

```
    double matriz[8][10] = {0};
    double vector[8]= {0};
    string tipoSangre[] = {"A+","A-","B+","B-","AB+","AB-","O+","O-"};
    int total = auxDonor.size();

    for (const auto &d : auxDonor){
        if (d.bloodType >=1 && d.bloodType <= 8 && d.district >= 1 && d.district <=10){
            matriz[d.bloodType-1][d.district-1]++;
            vector[d.bloodType-1]++;
        }
    }
    cout<<"TS/DP\t";
    for (int j = 0; j <=9;j++){
        cout<<"D("<<j+1<<")\t";
    }
    cout<<"Total\t";
    cout<<"(%) TP\t"<<endl;

    for (int i = 0; i < 8;i++){
        cout<<tipoSangre[i]<<"\t";
        for (int j =0; j < 10;j++){
            cout<<matriz[i][j]<<"\t";
        }
        cout<<vector[i]<<"\t";
        cout<<(vector[i]/total)*100<<"%"<<endl;
    }
}
```

Adicional a esto, se corrigió el código para generar los reportes de la opción 4 que venían como sugerencia en el archivo main:

